

C# - PHẦN 9.3

ACTION VÀ ACTION<T>

Delegate có sẵn trong .NET cho các method trả về void

Ý chính của bài: Thay vì tự khai báo một Delegate trả về void, .NET đã cung cấp sẵn Action.

1. Nhắc lại Delegate cũ

Ở bài trước bạn có:

```
public delegate void NotificationDelegate(string message);
```

Sau đó:

```
NotificationDelegate notification =  
    NotificationService.SendEmail;  
  
notification("Hello");
```

Delegate trên có chữ ký:

```
nhận string  
trả về void
```

.NET đã có sẵn một Delegate đúng kiểu đó:

```
Action<string>
```

Nên ta có thể thay:

```
NotificationDelegate notification =  
    NotificationService.SendEmail;
```

bằng:

```
Action<string> notification =  
    NotificationService.SendEmail;
```

Kết quả vẫn giống nhau.

2. Action là gì?

Action là một Delegate có sẵn trong .NET dùng cho method không trả về giá trị. Tức là return type luôn là:

```
void
```

Ví dụ method:

```
public static void SayHello()  
{  
    Console.WriteLine("Hello");  
}
```

Ta có thể dùng:

```
Action action = SayHello;
```

Sau đó:

```
action();
```

Luồng:

```
action
↓
SayHello
↓
Hello
```

3. Action không có tham số

```
public static void ShowMessage()
{
    Console.WriteLine("Welcome!");
}
```

Dùng:

```
Action action = ShowMessage;

action();
```

Ở đây:

```
Action
→ không có parameter
→ return void
```

Có thể hiểu chữ ký là:

```
void Method()
```

4. Action<T> có một tham số

```
public static void PrintName(string name)
{
    Console.WriteLine($"Name: {name}");
}
```

Ta dùng:

```
Action<string> action = PrintName;
```

Sau đó:

```
action("Hung");
```

Ở đây Action<string> nghĩa là:

```
nhận string
trả void
```

Tương đương với việc bạn tự khai báo:

```
public delegate void MyDelegate(string value);
```

5. Action<T1, T2> có nhiều tham số

```
public static void DisplayStudent(  
    string name,  
    int age)  
{  
    Console.WriteLine(  
        $"Name: {name}, Age: {age}");  
}
```

Ta dùng:

```
Action<string, int> display =  
    DisplayStudent;
```

Sau đó:

```
display("Hung", 20);
```

Đọc Action<string, int> là: Delegate nhận string, int và trả về void.

Tương đương:

```
public delegate void StudentDelegate(  
    string name,  
    int age);
```

6. Quy tắc đọc Action

Action

→ void Method()

Action<string>

→ void Method(string)

Action<string, int>

→ void Method(string, int)

Action<int, int, string>

→ void Method(int, int, string)

Điểm cần nhớ: Tất cả các kiểu nằm trong <...> đều là kiểu parameter, không có kiểu return vì Action luôn trả void.

7. Ví dụ với bài Notification của bạn

Bạn vừa làm:

```
public delegate void NotificationDelegate(  
    string message);
```

Thực ra có thể bỏ Delegate đó và viết:

```
Action<string> notification =  
    NotificationService.SendEmail;
```

Sau đó Multicast vẫn hoạt động:

```
notification += NotificationService.SendSms;
notification += NotificationService.WriteLog;

notification("Order completed");
```

Và:

```
notification -= NotificationService.SendSms;

notification("Payment completed");
```

Action<T> cũng là Delegate nên nó vẫn hỗ trợ =, +=, -= và multicast.

8. Vậy Custom Delegate có còn cần không?

Có. Hai cách sau về mặt chữ ký rất giống nhau:

```
public delegate void NotificationDelegate(
    string message);

Action<string>
```

Custom Delegate có lợi khi bạn muốn tên Delegate thể hiện ý nghĩa nghiệp vụ, ví dụ:

```
OrderCompletedHandler
StudentRegisteredHandler
PaymentProcessedHandler
```

Trong khi Action<string> chỉ cho biết nhận string và trả void, chứ không nói string đó đại diện cho cái gì.

Hiện tại chỉ cần hiểu: Custom Delegate = tự định nghĩa tên và chữ ký; Action = Delegate có sẵn của .NET cho method trả void.

9. Action có thể được truyền vào method

Đây chính là kiến thức Delegate cũ.

```
public static void Execute(Action action)
{
    Console.WriteLine("Before");

    action();

    Console.WriteLine("After");
}
```

Method:

```
public static void SayHello()
{
    Console.WriteLine("Hello");
}
```

Gọi:

```
Execute(SayHello);
```

Luồng:

```
Execute(SayHello)
```

↓

```
Before
```

↓

```
action()
```

↓

```
SayHello()
```

↓

```
Hello
```

↓

```
After
```

Bạn thấy nó giống bài trước:

```
Calculate(
```

```
    10,
```

```
    5,
```

```
    Calculator.Add);
```

Chỉ khác ở đây Delegate là Action.

10. Action<T> truyền vào method

```
public static void ProcessMessage(  
    string message,  
    Action<string> handler)  
{  
    handler(message);  
}
```

Có:

```
public static void PrintMessage(string message)  
{  
    Console.WriteLine(message);  
}
```

Gọi:

```
ProcessMessage(  
    "Hello",  
    PrintMessage);
```

Trong method, handler(message) thực chất là:

```
PrintMessage("Hello");
```

11. Tại sao Action hữu ích?

Giả sử một method cần làm công việc nào đó, nhưng hành vi cụ thể do người gọi quyết định.

```
public static void Process(  
    string value,  
    Action<string> action)  
{
```

```
    action(value);  
}
```

Method này không quan tâm action là Print, Save, Log hay Notify, miễn là method đó có chữ ký:

```
void Method(string)
```

Ví dụ:

```
Process(  
    "YC01",  
    NotificationService.SendEmail);  
  
Process(  
    "YC01",  
    NotificationService.WriteLog);
```

12. So sánh Custom Delegate và Action

Custom Delegate	Action
Tự khai báo	.NET cung cấp sẵn
Tự đặt tên	Tên cố định là Action
Có thể trả void hoặc giá trị khác	Luôn trả void
Có thể mô tả ý nghĩa nghiệp vụ rõ	Ngắn gọn, tiện sử dụng
Ví dụ NotificationDelegate	Ví dụ Action<string>

Ví dụ:

```
public delegate void NotificationDelegate(  
    string message);
```

tương đương về chữ ký với:

```
Action<string>
```

13. Điều cực kỳ quan trọng

Action<T> không phải Generic Method.

Ví dụ:

```
Action<string>
```

Action<T> là một Generic Delegate có sẵn của .NET. T trong Action<T> đại diện cho kiểu parameter.

```
Action<int>  
→ method nhận int
```

```
Action<Product>  
→ method nhận Product
```

```
Action<string, decimal>  
→ method nhận string và decimal
```

Đừng nhầm nó với:

```
public static void DisplayValue<T>(T value)
```

Đây mới là Generic Method.

14. Ví dụ gần với Collections

```
List<string> names =  
    new List<string>  
    {  
        "Hung",  
        "An",  
        "Lam"  
    };
```

Ta có:

```
Action<string> displayName =  
    PrintName;
```

với:

```
public static void PrintName(string name)  
{  
    Console.WriteLine(name);  
}
```

Sau đó:

```
foreach (string name in names)  
{  
    displayName(name);  
}
```

Kết quả:

```
Hung  
An  
Lam
```

Sau này .NET cho phép viết ngắn hơn rất nhiều bằng Lambda. Nhưng hiện tại chưa cần.

15. Ví dụ với object

```
public class Student  
{  
    public string Name { get; }  
  
    public Student(string name)  
    {  
        Name = name;  
    }  
}
```

Method:

```
public static void DisplayStudent(Student student)  
{
```

```
Console.WriteLine(student.Name);  
}
```

Delegate:

```
Action<Student> display =  
    DisplayStudent;
```

Gọi:

```
Student student = new Student("Hung");  
  
display(student);
```

Ở đây:

```
Action<Student>  
→ nhận Student  
→ trả void
```

16. Action có thể Multicast

Vì nó là Delegate nên:

```
Action<string> notification =  
    NotificationService.SendEmail;  
  
notification += NotificationService.SendSms;  
notification += NotificationService.WriteLog;
```

hoàn toàn hợp lệ. Sau đó:

```
notification("Hello");
```

gọi cả ba method. Kiến thức 9.2 được giữ nguyên.

17. Tóm tắt bài 9.3

```
Action  
→ Delegate có sẵn  
→ không parameter  
→ trả void  
  
Action<T>  
→ Delegate có sẵn  
→ nhận 1 parameter kiểu T  
→ trả void  
  
Action<T1, T2>  
→ nhận 2 parameter  
→ trả void
```

Ví dụ:

```
Action<string> action = PrintName;
```

nghĩa là PrintName phải nhận string và trả void.

Nguyên tắc quan trọng nhất: Action luôn trả void.

Bài tập 9.3 - Action

Tạo project/file theo cấu trúc:

```
ActionBasic/  
├─ NotificationService.cs  
├─ ActionProcessor.cs  
└─ Program.cs
```

NotificationService

Có 3 method sau, đều trả void:

```
SendEmail(string message)  
SendSms(string message)  
WriteLog(string message)
```

Bạn có thể dùng lại code bài 9.2.

ActionProcessor

Tạo:

```
public static class ActionProcessor  
{  
    public static void Process(  
        string message,  
        Action<string> action)  
    {  
        // gọi action với message  
    }  
}
```

Bạn tự hoàn thiện phần thân.

Program

Thực hiện các trường hợp sau:

1. Khai báo Action<string> notification trỏ tới SendEmail.
2. Gọi notification("Hello").
3. Đổi notification sang SendSms rồi gọi lại.
4. Thêm WriteLog bằng += để kiểm tra multicast.
5. Gọi ActionProcessor.Process("Order completed", NotificationService.SendEmail).
6. Gọi Process() lần nữa, nhưng truyền WriteLog.

Câu hỏi cuối bài

7. Action khác Custom Delegate ở đâu?
8. Action<string> yêu cầu method có chữ ký như thế nào?
9. Vì sao Action không dùng được với method trả về int hoặc decimal?
10. Action<string> có hỗ trợ += và -= không? Vì sao?

Mẹo học: Mỗi khi thấy Action<string>, hãy tự dịch ngay trong đầu thành “một Delegate nhận string và trả void”. Nếu làm được phản xạ đó thì bài 9.3 gần như đã hiểu.